

LLMOps Portfolio Project — Deliverables Checklist

Scaler School of Technology | ML System Design and LLMOps | Capstone Submission Guide



This checklist translates the guiding principles in `llmops_portfolio_projects_handout.pdf` and the grading criteria in `presentation_rubric.pdf` into a concrete list of what to hand in. If an item doesn't apply to your chosen project, say so explicitly in your README rather than skipping it silently.

Note: Deadlines and grading weights are set by the instructor and shared separately — this document covers **what** to submit, not **when**.

Grading is a group effort, but not a flat split. The project is built and graded as a team, so all members will receive similar marks — but not necessarily identical ones. Individual performance during the presentation and Q&A (per `presentation_rubric.pdf`) can shift a member's marks slightly up or down relative to the rest of the team.

What to Submit

1. Live deployment (*good to have, not mandatory*)

- A live, public URL a recruiter can click without setup (Fly.io / Render / HF Spaces free tier, or equivalent).
- The deployed app matches the code on your repo's main branch.

2. GitHub repository

- Public repo with a clear name and description.
- README.md containing: problem statement; architecture overview; **numbers** (latency p50/p99, cost per request, eval scores, load-test throughput — no numbers, no credit); setup instructions; link to the live deployment.
- Prompt/config versioned in-repo, not hardcoded where it can't be diffed.

3. Evaluation set

- One hand-written, domain-specific eval set (no toy/public-benchmark copy-paste). Size depends on project type — e.g. 30–50 Q&A pairs for RAG, ~20 tasks for agents, ~50 adversarial prompts for guardrails.
- Committed to the repo with the scoring method documented (retrieval metrics, LLM-as-judge, programmatic checks, as applicable).

4. Observability (*if applicable*)

- Cost and latency tracked per request.
- A dashboard (Grafana / Streamlit or equivalent) or logged traces (Langfuse / Arize Phoenix or equivalent) showing this data.

5. Presentation

- 15-minute presentation + 5-minute Q&A, scored against `presentation_rubric.pdf`: problem framing (20%), architecture design (30%), LLMOps depth (20%), trade-off reasoning (20%), presentation quality (10%).
- Team is prepared to complete a peer review sheet for other teams' presentations — see Section 7. This is participation-graded, not optional.

6. Resume line

- A 2–3 sentence resume-ready description of what you built, styled like the examples in the project handout.

Peer Review Policy

7. Peer review (*part of your participation grade*)

- Every team is reviewed by **two other teams**. Reviewing teams are announced before the presentation date.
- Each reviewing team receives a Peer Review Grading Sheet for its assigned team.
- Complete the review during the presentation and Q&A — have the sheet open from the start and write as you go. The submission window is for *finishing* your review, not starting it from scratch.
- Submit the sheet within **10 minutes** of the presentation ending. Late submissions will not be accepted.
- Reviewer names are removed before the sheet is shared with the presenting team. The instructor reads every submitted sheet.

Review quality matters. Reviews must be genuine, critical, specific, and based on what you actually observed — generic or superficial feedback is not sufficient.

- AI-generated, copied, superficial, or non-genuine reviews will be **penalised with negative marks** on the reviewing team's project grade.
- Reviews that fail to demonstrate genuine critical engagement with the system may similarly receive negative marks.
- If a review is substantially different from the instructor's own review in a way that indicates the reviewing team misunderstood, overlooked, or failed to critically evaluate important aspects of the system, negative marks may be assigned to the reviewing team's project grade.
- The strongest peer-review questions may be brought into that team's Q&A.

Critically reviewing someone else's architecture is as valuable an ML system-design skill as designing your own.

Notes

- **Budget guidance:** ~ \$5–\$20 of API credits total. Use Gemini Flash / Claude Haiku / GPT-4o-mini as the default model and as the LLM-judge; cache aggressively in dev.
- **Not building one of the 12 menu projects?** The same checklist still applies — every item above is a guiding principle from the handout, not specific to any one project.